

Part 01 – Theoretical Questions

Question 1: What is the purpose of the `finally` block?

Answer:

The `finally` block is used to execute a block of code regardless of whether an exception occurs or not. It is commonly used for cleanup operations, such as closing files, database connections, or releasing resources. The code inside the `finally` block is executed after the `try` and `catch` blocks.

Question 2: How does `int.TryParse()` improve program robustness compared to `int.Parse()`?

Answer:

`int.TryParse()` improves program robustness because it does not throw an exception when the input is invalid. Instead, it returns `true` if the conversion is successful and `false` if the input cannot be converted to an integer. This allows the program to handle invalid user input safely without crashing.

Question 3: What exception occurs when trying to access `Value` on a null `Nullable<T>`?

Answer:

When the `Value` property of a `Nullable<T>` is accessed while it contains `null`, an `InvalidOperationException` is thrown.

The `HasValue` property should be checked before accessing `Value` to make sure that the nullable variable actually contains a value.

Question 4: Why is it necessary to check array bounds before accessing elements?

Answer:

It is necessary to check array bounds to make sure that the requested index is within the valid

range of the array. Accessing an index outside the valid range causes an `IndexOutOfRangeException`, which can terminate the program if it is not handled.

For an array with five elements, the valid indexes are from 0 to 4.

Question 5: How is the `GetLength(dimension)` method used in multi-dimensional arrays?

Answer:

The `GetLength(dimension)` method returns the number of elements in a specific dimension of a multi-dimensional array.

For example, in a 3×3 array:

- `GetLength(0)` returns the number of rows.
- `GetLength(1)` returns the number of columns.

It is useful when writing loops because it allows the program to determine the size of each dimension dynamically.

Question 6: How does the memory allocation differ between jagged arrays and rectangular arrays?

Answer:

A rectangular array is a single multi-dimensional array where all rows have the same number of elements. Its elements are stored as one rectangular structure.

A jagged array is an array of arrays. Each row is a separate array, so each row can have a different size. Therefore, the rows are allocated separately in memory.

Question 7: What is the purpose of nullable reference types in C#?

Answer:

Nullable reference types help developers identify and handle possible `null` values in reference types. By declaring a reference type as nullable using `?`, the compiler can warn the developer when the value might be `null`.

This helps reduce the possibility of `NullReferenceException` and makes the code safer and easier to understand.

Question 8: What is the performance impact of boxing and unboxing in C#?

Answer:

Boxing and unboxing can have a performance cost. Boxing converts a value type into an object, which usually requires allocating memory on the heap and copying the value. Unboxing requires checking the stored type and converting the object back to the value type.

Frequent boxing and unboxing can increase memory usage and reduce performance, especially when they occur repeatedly inside loops or performance-sensitive code.

Question 9: Why must out parameters be initialized inside the method?

Answer:

out parameters must be assigned a value inside the method before the method returns because the caller is not required to initialize them before passing them to the method.

The compiler ensures that every out parameter receives a value before the method finishes. This guarantees that the caller receives valid output values.

Question 10: Why must optional parameters always appear at the end of a method's parameter list?

Answer:

Optional parameters must appear at the end of the parameter list because required parameters must be provided before optional parameters. If an optional parameter appeared before a required parameter, it could create ambiguity about which argument belongs to which parameter.

For example, this is valid:

```
void Test(string name, int age = 20)
```

But an optional parameter should not be placed before a required parameter.

Question 11: How does the null propagation operator prevent `NullPointerException`?

Answer:

The null propagation operator `?.` safely accesses a member of an object only if the object is not `null`. If the object is `null`, the expression returns `null` instead of trying to access the member.

For example:

```
int? length = numbers?.Length;
```

If `numbers` is `null`, no exception is thrown and `length` becomes `null`.

Question 12: When is a switch expression preferred over a traditional `if` statement?

Answer:

A switch expression is preferred when a single value needs to be compared with multiple possible patterns or values and each case produces a corresponding result. It is usually more concise and readable than a long chain of `if-else` statements.

For example, a day of the week can be mapped to its number using a switch expression.

Question 13: What are the limitations of the `params` keyword in method definitions?

Answer:

The `params` keyword has several limitations:

1. A method can have only one `params` parameter.
2. The `params` parameter must be the last parameter in the method.
3. The `params` parameter must be a one-dimensional array.
4. All values passed to the `params` parameter must be compatible with its element type.
5. The method can be called either with individual values or with an array.